



BLUE NOVA SAS

BASTION

Boveda Personal de Credenciales

GUION DEL VIDEO

David Alexander Benz Zambrano

Logica de Programacion - UIDE - Paralelo 1-CIB-1A

Quito - Ecuador - 2026

Guion del video demostrativo - Proyecto BASTION

Formato de grabacion: Loom con captura de pantalla y voz. Duracion objetivo: 5 a 7 minutos. Narracion en primera persona, voz de David. Habla pausado, no leas de corrido. Las marcas de tiempo son aproximadas y sirven de ritmo, no de cronometro estricto.

Antes de empezar a hablar, ten ya abiertas en pestanas separadas: el repositorio en GitHub, la terminal con la carpeta del proyecto, y la aplicacion web en su URL. Asi no pierdes segundos buscando ventanas.

Escena 1 - Introduccion (00:00 a 00:40)

En pantalla: - Pantalla de portada o el repositorio de GitHub abierto con el nombre BASTION visible. - Opcional: tu camara en la esquina por un par de segundos para dar la cara.

Narracion sugerida: "Hola, mi nombre es David Alexander Benz Zambrano, soy estudiante de Ingenieria en Ciberseguridad en la UIDE, paralelo uno C I B uno A. En este video les presento mi proyecto integrador de Logica de Programacion: se llama BASTION, una boveda personal de credenciales. La idea es sencilla de explicar pero util de verdad: un programa donde guardo mis contraseñas de forma ordenada y protegida, y que ademas me genera contraseñas fuertes cuando las necesito. Lo construi en dos versiones, una de consola en Python puro y una aplicacion web, y en los proximos minutos les muestro las dos funcionando."

Escena 2 - El problema que resuelve (00:40 a 01:25)

En pantalla: - Una imagen o diapositiva simple con el problema: muchas cuentas, contraseñas repetidas, papelitos o notas en el telefono. - O simplemente tu cara o el escritorio mientras explicas.

Narracion sugerida: "Todos tenemos decenas de cuentas: redes sociales, banca, correo, trabajo. Y por comodidad caemos en lo mismo: repetir la misma contraseña en todos lados, o anotarlas en una nota del telefono sin ninguna proteccion. Eso es justo lo que un atacante necesita, porque si le roban una sola clave, entra a todo. BASTION ataca ese problema desde la ciberseguridad: una contraseña maestra protege toda la boveda, esa maestra nunca se guarda en texto plano sino como un hash SHA-256, y el acceso se bloquea despues de tres intentos fallidos. Ademas genera contraseñas largas y aleatorias, distintas para cada servicio, así ya no hay excusa para repetir las. Y un detalle clave: los datos nunca salen de mi equipo ni del navegador, no viajan a ninguna nube, lo que reduce la superficie de ataque."

Escena 3 - El repositorio en GitHub (01:25 a 02:15)

En pantalla: - El repositorio `github.com/Bluenovasas/bastion-uide` abierto. - Recorre las carpetas con el cursor: el código Python, la carpeta webapp, el README, la licencia. -

Abre la pestaña de commits y haz zoom en la insignia verde "Verified" de un commit firmado.

Narracion sugerida: "Este es el repositorio del proyecto en GitHub. Lo organice por carpetas para que se entienda la arquitectura: aquí está el archivo principal bastion punto pe y, en la carpeta webapp, está la aplicación web con su index, sus estilos y su JavaScript. También incluyo el README con la documentación académica y la licencia. Quiero detenerme en los commits: como ven, cada commit tiene la insignia Verified, en verde. Eso significa que están firmados criptográficamente con mi llave, así se garantiza que ese código realmente lo subí yo y no fue alterado. Para una carrera de ciberseguridad eso no es un adorno, es parte de la cultura de integridad del código."

Escena 4 - El código Python bastion.py (02:15 a 03:45)

En pantalla: - Abre bastion.py en el editor. - Mientras nombras cada estructura, haz scroll y selecciona o resalta el bloque exacto: el while con contador en iniciar_sesion, el for con range en generar_contrasena, el for con .items() en ver_credenciales o buscar_credencial, el if/elif/else en calcular_fortaleza, la tupla CATEGORIAS arriba, la lista historial dentro de la boveda, y la firma de una función con parametro por defecto. - Luego pasa a la terminal y ejecuta python3 bastion.py por unos segundos.

Narracion sugerida: "Vamos al código. Este es bastion punto pe, escrito solo con la biblioteca estándar de Python. Aquí está el corazón del proyecto, donde se ven las estructuras de las cuatro unidades del curso. Primero, en iniciar sesion uso un bucle while con un contador de intentos: arranca en cero, sube de uno en uno con el operador mas igual, y a los tres intentos fallidos bloquea la boveda. Aquí abajo, en el generador, uso un for con range para construir la contraseña carácter por carácter, repitiendo tantas veces como la longitud que pida el usuario. Mas alla, para listar y buscar credenciales, recorro el diccionario con un for usando punto items, que me da a la vez el servicio y sus datos. La fortaleza de la contraseña la decido con un if, elif, else: debil, media o fuerte segun la longitud y los tipos de caracteres. Arriba defini las categorias como una tupla, redes sociales, banca, trabajo, correo, claves A P I o S S H, y otros; es una tupla justamente porque las categorias no deben cambiar en tiempo de ejecucion, son inmutables. La boveda tambien guarda una lista de historial con cada acceso y su fecha y hora. Y tengo funciones con parametros por defecto, por ejemplo registrar acceso, que recibe un evento con un valor por defecto, asi puedo llamarla sin pasarle todo. Ahora lo ejecuto en consola para que vean que corre de verdad: aqui crea la boveda, pide la maestra, genero una contraseña, la guardo, y la listo enmascarada. Todo funcionando."

Escena 5 - La aplicación web en vivo (03:45 a 05:45)

En pantalla: - Abre la URL <https://bluenovasas.github.io/bastion-ui/webapp/> en el navegador. - Crea una boveda con una contraseña maestra. - Genera una contraseña segura (ajusta la longitud y marca mayúsculas, números, símbolos; muestra el indicador Debil/Media/Fuerte y el check "Mostrar contraseña"). - Agrega una credencial eligiendo categoría, usuario y contraseña. - Busca esa credencial por nombre. - Abre el historial de

accesos con fecha y hora. - **IMPORTANTE:** durante todo esto, senala con el cursor el panel "Consola Python" que se va llenando con cada llamada.

Narracion sugerida: "Y esta es la aplicacion web, publicada con GitHub Pages, con la identidad visual de Blue Nova. Lo interesante es que no reescribi el programa en JavaScript: esta web carga Pyodide, que es Python compilado a WebAssembly, y ejecuta el mismo bastion punto pe dentro del navegador. Lo demuestro. Creo la boveda con mi contrasena maestra. Ahora genero una contrasena: subo la longitud, activo mayusculas, numeros y simbolos, y miren como el indicador me dice si es debil, media o fuerte; con este check de Mostrar contrasena puedo ver lo que escribo. Agrego una credencial: elijo la categoria, pongo el usuario y guardo. Ahora la busco por su nombre, sin importar mayusculas, y ahi aparece. Y aqui esta el historial: cada acceso queda registrado con su fecha y hora. Fijense en este panel de la derecha, la Consola Python: cada accion que hago en la interfaz dispara una llamada a una funcion real de Python, y aqui se imprime en vivo. Esa es la prueba de que el codigo Python se ejecuta de verdad, no es una imitacion en JavaScript. Aqui se ven las tres capas con total claridad: la presentacion es esta interfaz web, la logica de negocio es bastion punto pe corriendo en Pyodide, y la persistencia es el almacenamiento local del navegador. Y lo mejor: la docente puede entrar a esta misma URL desde su computadora y probarla ella misma, sin instalar nada."

Escena 6 - Reflexion y cierre (05:45 a 06:40)

En pantalla: - Vuelve a la pagina principal del proyecto o a una diapositiva de cierre con tu nombre y el de la materia. - Opcional: tu camara de nuevo para cerrar dando la cara.

Narracion sugerida: "Para cerrar, una reflexion. La tecnologia hoy nos pide una identidad digital cada vez mas grande, y con ella crece el riesgo. Una sola contrasena debil o repetida puede comprometer la vida entera de una persona: su dinero, su privacidad, su trabajo. Herramientas como BASTION democratizan la seguridad, porque ponen al alcance de cualquiera buenas practicas que antes parecian solo para expertos: contrasenas fuertes, cifrado y datos que no se entregan a terceros. Como futuro ingeniero en ciberseguridad, creo que nuestra responsabilidad no es solo proteger sistemas, sino construir tecnologia que respete y empodere a las personas. BASTION es un proyecto academico, todavia con limitaciones, pero su evolucion ya esta pensada con cifrado A E S doscientos cincuenta y seis G C M y derivacion de clave Argon2. Gracias por ver el video, soy David Benz, y este fue mi proyecto integrador BASTION."

Lista de verificacion previa a la grabacion

Tecnica

- [] Loom instalado, con permiso de microfono y de captura de pantalla concedidos.
- [] Microfono probado: graba 10 segundos de prueba y escucha el volumen y el ruido de fondo.

- [] Resolucion de pantalla y tamano de fuente del editor y la terminal aumentados para que el codigo se lea bien.
- [] Modo "no molestar" activado: silencia notificaciones de correo, WhatsApp, Slack y del sistema.
- [] Buena conexion a internet (la web descarga Pyodide la primera vez; cargala una vez antes de grabar para que quede en cache).

Contenido y pestanas listas

- [] Repositorio github.com/Bluenovasas/bastion-ui abierto, con la vista de commits firmados a un clic.
- [] Editor con `bastion.py` abierto y posicionado al inicio.
- [] Terminal abierta en la carpeta del proyecto, lista para `python3 bastion.py`.
- [] Aplicacion web cargada y probada en la URL real, con la Consola Python visible.
- [] Diapositiva o portada de introduccion y de cierre preparada (opcional pero recomendable).

Datos de demostracion

- [] Define de antemano la contrasena maestra de prueba (que la recuerdes y la sepas teclear sin titubear).
- [] Prepara el nombre del servicio, usuario y categoria que vas a agregar en la demo.
- [] Si la boveda ya tiene datos viejos, decide si la limpias antes para que la demo se vea fresca (en consola, respaldar o borrar `boveda.json`; en la web, limpiar el almacenamiento local).
- [] No muestres en pantalla ninguna contrasena real tuya: usa siempre datos ficticios de prueba.

Ensayo y forma

- [] Ensayo el guion al menos una vez completo y cronometra que quede entre 5 y 7 minutos.
- [] Practica el orden de la demo web para no dudar: crear, generar, agregar, buscar, historial.
- [] Ten claro donde esta cada estructura de codigo para resaltarla sin buscarla en vivo.
- [] Habla pausado, con energia, y mira hacia la pantalla que estas mostrando, no a otra parte.
- [] Al terminar, revisa la grabacion completa antes de subirla: audio claro, pantalla legible y nada confidencial visible.